EUROPEAN UNIVERSITY OF LEFKE
Faculty of Engineering
Department of Software Engineering




# COMP 337

DATABASE MANAGEMENT SYSTEMS

# Lab Manual 1

*Banking Management System (Python & MySQL)*

Prepared by: Yordanos Mezgebe Kahsay (2503060413)

Submitted to Mr. Salman Khan

# Contents

1. Introduction and objectives

2. Tools and requirements

3. Database design

4. System architecture

5. Implementation

6. User interfaces

7. Testing

8. Sample run

9. Installation and running

10. Conclusion

Appendix A. Full source code

# 1. Introduction and objectives

This lab work implements a **Banking Management System**: a Python application that stores customers, their bank accounts and every money movement in a MySQL database. A bank clerk can register customers, open accounts, deposit, withdraw and transfer money, and look at balances, transaction history and a summary of the whole bank.

The objectives of the lab were to:

1. design a normalised relational schema with primary keys, foreign keys and integrity constraints;
2. connect a Python program to MySQL and run parameterised SQL (INSERT, SELECT, UPDATE, DELETE, JOIN, GROUP BY);
3. use **database transactions** so that multi-step operations (such as a transfer) are atomic;
4. validate user input and handle errors without crashing; and
5. give the system a usable interface: a text menu and a Tkinter desktop window.

The complete source code is published at **https://github.com/Yordikah/Banking-management-system**.

# 2. Tools and requirements

| Component | Used for |
|---|---|
| Python 3.9+ | Application language |
| MySQL 8.0 / MySQL Workbench | Database server and administration |
| mysql-connector-python | Official MySQL driver for Python |
| Tkinter (standard library) | Desktop graphical interface |
| unittest (standard library) | Automated tests |
| Visual Studio Code, Git, GitHub | Editing and version control |

## Functional requirements

| Area | The system must... |
|---|---|
| Users | add, list, search by name, edit and delete customers (a customer who still has accounts cannot be deleted) |
| Accounts | open savings, checking or business accounts with an opening balance; list accounts; show a balance; close an account only when its balance is zero |
| Transactions | deposit, withdraw and transfer money; never allow a balance below zero; record every movement in the history |
| Reports | show the number of customers and accounts, total deposits, and totals per account type |

# 3. Database design

The database `banking_app` has three tables. One customer can own many accounts, and one account can have many transactions, giving two one-to-many relationships.

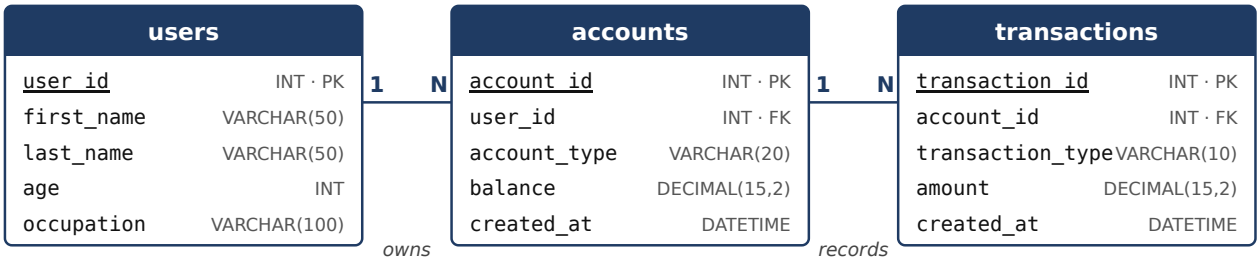



*Figure 1. Entity–relationship diagram of the banking_app database*

## 3.1 Tables and constraints

| Table | Primary / foreign key | Integrity rules |
|---|---|---|
| `users` | PK `user_id` (auto-increment) | Names required; `CHECK (age BETWEEN 18 AND 120)` |
| `accounts` | PK `account_id`; FK `user_id → users` | `CHECK (balance >= 0)`; type limited to savings / checking / business; `created_at` set automatically |
| `transactions` | PK `transaction_id`; FK `account_id → accounts` | Type is `credit` or `debit`; `CHECK (amount > 0)`; index on (account_id, created_at) for fast history |

## 3.2 Design decisions

- **DECIMAL(15,2) for money.** The first version stored amounts as `DOUBLE`. Floating-point numbers cannot represent most decimal fractions exactly ($0.1 + 0.2 \neq 0.3$), which is unacceptable for balances. DECIMAL stores exact cents, and the Python code uses `decimal.Decimal` to match.
- **Normalisation.** Customer details are stored once in `users`; accounts and transactions refer to them by key, so the schema is in third normal form and has no update anomalies.
- **Constraints in the database, not only in Python.** Even if a bug or another tool writes to the database, a negative balance or an orphaned account is rejected by MySQL itself.
- **Audit trail.** An opening balance is recorded as a first `credit`, so for every account the credits minus the debits always equal the balance.

## 3.3 SQL schema

```sql
-- Banking Management System - database schema (MySQL 8+)
-- Run with:  mysql -u root -p < sql/schema.sql

CREATE DATABASE IF NOT EXISTS banking_app;
```

```sql
USE banking_app;

CREATE TABLE IF NOT EXISTS users (
    user_id     INT AUTO_INCREMENT PRIMARY KEY,
    first_name  VARCHAR(50)  NOT NULL,
    last_name   VARCHAR(50)  NOT NULL,
    age         INT          NOT NULL,
    occupation  VARCHAR(100),
    CONSTRAINT chk_users_age CHECK (age BETWEEN 18 AND 120)
);

CREATE TABLE IF NOT EXISTS accounts (
    account_id    INT AUTO_INCREMENT PRIMARY KEY,
    user_id       INT            NOT NULL,
    account_type  VARCHAR(20)    NOT NULL,
    balance       DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
    created_at    DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_accounts_user FOREIGN KEY (user_id) REFERENCES users (user_id),
    CONSTRAINT chk_accounts_balance CHECK (balance >= 0),
    CONSTRAINT chk_accounts_type CHECK (account_type IN ('savings', 'checking', 'business'))
);

CREATE TABLE IF NOT EXISTS transactions (
    transaction_id    INT AUTO_INCREMENT PRIMARY KEY,
    account_id        INT            NOT NULL,
    transaction_type  VARCHAR(10)    NOT NULL,
    amount            DECIMAL(15, 2) NOT NULL,
    created_at        DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transactions_account FOREIGN KEY (account_id) REFERENCES accounts (account_id),
    CONSTRAINT chk_transactions_type CHECK (transaction_type IN ('credit', 'debit')),
    CONSTRAINT chk_transactions_amount CHECK (amount > 0),
    INDEX idx_transactions_account (account_id, created_at)
);
```

# 4. System architecture

The program is split into layers, so that the database logic is written once and shared by both interfaces:

| Layer / file | Responsibility |
|---|---|
| `main.py` | Entry point. Checks the database connection, then starts the text menu or, with `--gui`, the window. |
| `banking/cli.py`, `banking/gui.py` | User interfaces. They only collect input and display results. |
| `banking/services/` | Business rules for users, accounts and transactions. All SQL lives here. |
| `banking/validation.py` | Parses and checks every input (IDs, ages, amounts, account types). |
| `banking/database.py` | Opens connections and runs each operation inside one transaction. |
| `banking/config.py` | Reads the database settings from a `.env` file, so the password is not stored in the code. |
| `banking/errors.py` | Error types shown to the user as friendly messages. |

The request flow is: *interface* → *validation* → *service* → *database*. Any problem raises a `BankingError`, which both interfaces catch and show to the user; the program never crashes with a Python traceback because of bad input.

## 4.1 Improvements over the first version

The first version of the project (initial draft) worked for simple cases but had several problems, which this version fixes:

| Problem in the first version | Fix |
|---|---|
| Withdrawals checked the balance and then updated it in separate steps, so two clerks withdrawing at the same time could both pass the check. The Workbench screenshot (Figure 2) shows a 7,500,000 debit recorded against account 1. | The balance row is locked with `SELECT … FOR UPDATE` before checking, and the check plus update run in one transaction. |
| Amounts used `float` in Python and `DOUBLE` in MySQL. | `Decimal` and `DECIMAL(15,2)`, with at most two decimal places allowed. |
| An error was raised with `BaseException`, which bypassed the `except Exception` handler and crashed the program. | Custom exception classes derived from `BankingError`. |
| Deposits to a non-existent account printed “Deposit successful!” even though nothing changed. | The account is looked up first; an unknown ID gives “Account N does not exist.” |

| Problem in the first version | Fix |
|---|---|
| No input checks: negative amounts, text in number fields, ages like 3. | Validation module covering every field (Section 5.3). |
| The database password was written in the source code. | Credentials come from a git-ignored `.env` file. |
| `view_users` was duplicated in two files; the menu had no transfer, history or account listing. | One service per entity; the menu now has 13 operations. |

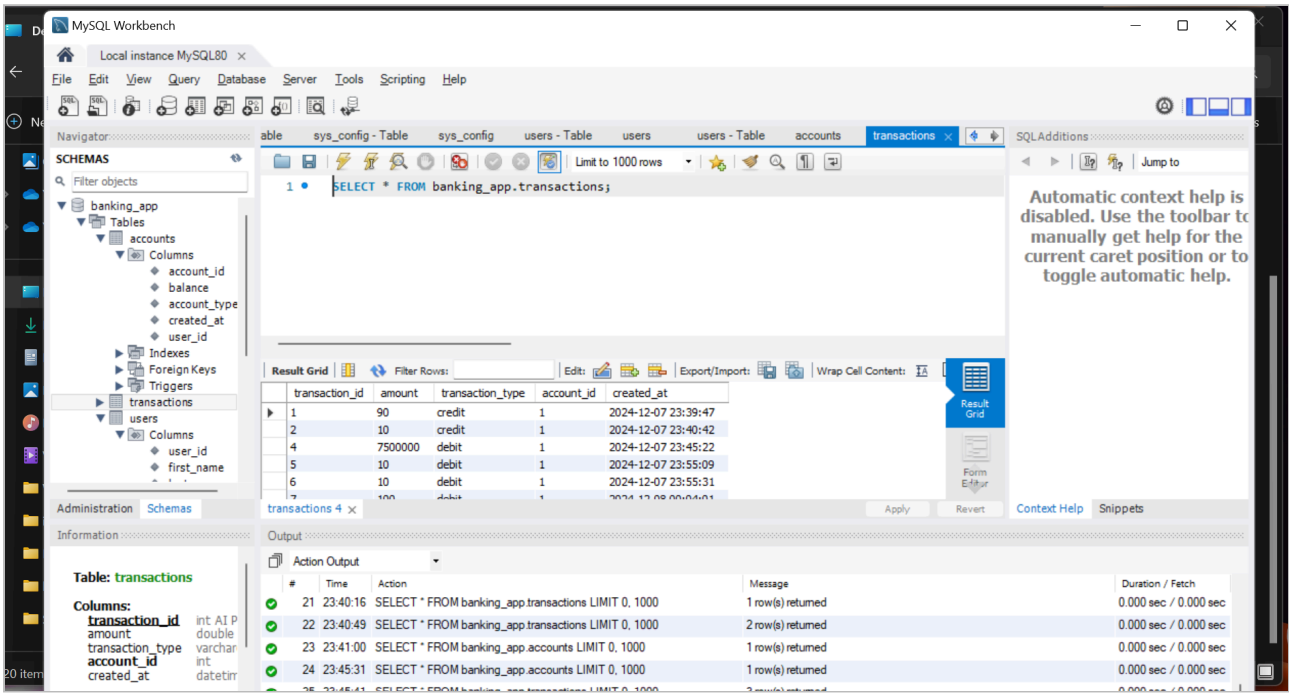



*Figure 2. The banking_app database in MySQL Workbench (first version, DOUBLE amount column)*

# 5. Implementation

## 5.1 Connection and transactions

`db_cursor()` is a context manager: it opens a connection, gives the caller a cursor, commits when the block finishes and **rolls back** if any exception occurs. Every service function does all of its SQL inside one `with db_cursor()` block, so it either completes fully or leaves the database unchanged.

```python
@contextmanager
def db_cursor():
    """Yield a dictionary cursor inside one transaction.

    Commits when the block finishes, rolls back on any exception, and always
    closes the connection. Every multi-step operation (e.g. a transfer) runs
    inside a single `with db_cursor()` block so it is all-or-nothing.
    """
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    try:
        yield cursor
        conn.commit()
    except Exception as exc:
        conn.rollback()
        _reraise_mysql_error(exc)
        raise
    finally:
        cursor.close()
        conn.close()
```

## 5.2 Withdrawal and transfer

Before checking a balance, the account row is locked with `SELECT … FOR UPDATE`; another transaction that wants the same row must wait until this one commits. A transfer locks both accounts in ascending ID order, so two opposite transfers cannot deadlock each other. The debit on the sender and the credit on the receiver are in the same transaction: if anything fails, neither happens.

```python
def _lock_balance(cur, account_id: int) -> Decimal:
    cur.execute("SELECT balance FROM accounts WHERE account_id = %s FOR UPDATE", (account_id,))
    row = cur.fetchone()
    if row is None:
        raise NotFoundError(f"Account {account_id} does not exist.")
    return Decimal(str(row["balance"]))
```

```python
def transfer(from_account_id, to_account_id, amount) -> Decimal:
    """Move money between two accounts. Returns the sender's new balance."""
    src = parse_id(from_account_id, "From account ID")
    dst = parse_id(to_account_id, "To account ID")
    value = parse_amount(amount, "Transfer amount")
    if src == dst:
        raise ValidationError("Cannot transfer to the same account.")

    with db_cursor() as cur:
        # Lock both rows in a fixed order to avoid deadlocks.
        balances = {aid: _lock_balance(cur, aid) for aid in sorted((src, dst))}
        if value > balances[src]:
            raise InsufficientFundsError(
```

```
                f"Insufficient funds: account {src} has {balances[src]:.2f}, "
                f"tried to transfer {value:.2f}."
            )
        _apply(cur, src, "debit", value)
        _apply(cur, dst, "credit", value)
        return balances[src] - value
```

## 5.3 Input validation

All input is checked before any SQL runs. Money is parsed with `Decimal`, must be positive, can have at most two decimal places and must fit in the column. All queries use `%s` parameters, never string formatting of user input, which prevents SQL injection.

```
def _parse_money(value, field: str) -> Decimal:
    try:
        amount = Decimal(str(value).strip().replace(",", ""))
    except (InvalidOperation, TypeError):
        raise ValidationError(f"{field} must be a number, e.g. 250 or 99.95.") from None
    if not amount.is_finite():
        raise ValidationError(f"{field} must be a number.")
    if amount.as_tuple().exponent < -2:
        raise ValidationError(f"{field} can have at most 2 decimal places.")
    if amount >= MAX_AMOUNT:
        raise ValidationError(f"{field} is too large.")
    return amount.quantize(Decimal("0.01"))


def parse_amount(value, field: str = "Amount") -> Decimal:
    """A transaction amount: strictly greater than zero."""
    amount = _parse_money(value, field)
    if amount <= 0:
        raise ValidationError(f"{field} must be greater than zero.")
    return amount
```

## 5.4 Queries used for listing and reports

The user list joins `users` with `accounts` and aggregates, so each customer is shown with the number of accounts and the total balance. `LEFT JOIN` keeps customers with no accounts, and `COALESCE` turns their missing sum into 0.

```
def list_users(search: str = "") -> list:
    """All users with how many accounts they hold and their total balance."""
    query = (
        "SELECT u.user_id, u.first_name, u.last_name, u.age, u.occupation, "
        "       COUNT(a.account_id) AS account_count, "
        "       COALESCE(SUM(a.balance), 0) AS total_balance "
        "FROM users u LEFT JOIN accounts a ON a.user_id = u.user_id "
    )
    params = ()
    term = (search or "").strip()
    if term:
        query += "WHERE u.first_name LIKE %s OR u.last_name LIKE %s "
        params = (f"%{term}%", f"%{term}%")
    query += "GROUP BY u.user_id ORDER BY u.user_id"

    with db_cursor() as cur:
        cur.execute(query, params)
        return cur.fetchall()
```

# 6. User interfaces

## 6.1 Text menu

Running `python main.py` opens the menu below. The options are grouped by area; after each operation the result or an error message is shown and the menu returns.

```
==========================================
        BANKING MANAGEMENT SYSTEM
==========================================

 Users
    1. Add user
    2. View / search users
    3. Edit user
    4. Delete user

 Accounts
    5. Create account
    6. View accounts
    7. Check balance
    8. Close account

 Transactions
    9. Deposit money
   10. Withdraw money
   11. Transfer money
   12. Transaction history

 Reports
   13. Bank summary

    0. Exit
```

## 6.2 Desktop window (Tkinter)

Running `python main.py --gui` opens a window with four tabs that use the same service layer:

- **Users** – a form (first name, last name, age, occupation) with Add, Update selected, Delete selected and Clear buttons; a live search box; and a table of all users with their account count and total balance. Clicking a row fills the form for editing.
- **Accounts** – choose the owner from a drop-down, the account type and the opening balance, then Create account; a table of all accounts with owner, type, balance and opening date; Close selected account.
- **Transactions** – pick an account (and a target for transfers), enter an amount and press Deposit, Withdraw or Transfer. The current balance and the account's history are shown underneath.
- **Summary** – cards with the number of users, accounts and total deposits, and a table of totals per account type.

Errors are shown in a message box and in the status bar; delete and close actions ask for confirmation first.

# 7. Testing

## 7.1 Automated unit tests

`tests/test_validation.py` contains 11 unit tests for the validation rules (text fields, IDs, ages, amounts, opening balances and account types). They need no database and are run with `python -m unittest -v`. Result: **11 tests, all passed.**

## 7.2 Functional test cases

The service layer was run end-to-end against a test database with the same three tables. Each case was checked for the expected result *and* for the balances afterwards.

| ID | Test | Input | Expected | Actual | Result |
|---|---|---|---|---|---|
| TC1 | Add user with valid data | Ayse, Demir, 29, Engineer | User saved, ID 1 returned | User added with ID 1 | **Pass** |
| TC2 | Add user under 18 | Age = 15 | Rejected | "Age must be between 18 and 120." | **Pass** |
| TC3 | Add user with empty name | First name = "" | Rejected | "First name is required." | **Pass** |
| TC4 | Open account for unknown user | User ID = 99 | Rejected | "User 99 does not exist. Add the user first." | **Pass** |
| TC5 | Open account with invalid type | Type = gold | Rejected | Lists the valid types | **Pass** |
| TC6 | Deposit | Account 1 (1,000.00), +500 | Balance 1,500.00 | Balance 1,500.00 | **Pass** |
| TC7 | Deposit negative / text | −5, "abc" | Rejected | Both rejected with a clear message | **Pass** |
| TC8 | Withdraw more than balance | Account 1 (1,500.00), −5,000 | Rejected, balance unchanged | Insufficient funds; balance still 1,500.00 | **Pass** |
| TC9 | Withdraw | Account 1, −200 | Balance 1,300.00 | Balance 1,300.00 | **Pass** |
| TC10 | Transfer | Account 1 → 2, 300 | 1,000.00 and 550.00 | 1,000.00 and 550.00 | **Pass** |
| TC11 | Transfer to same account | Account 1 → 1 | Rejected | "Cannot transfer to the same account." | **Pass** |
| TC12 | Transfer more than balance | Account 1 → 2, 9,999 | Rejected, both balances unchanged | Rolled back; 1,000.00 and 550.00 | **Pass** |
| TC13 | Delete user who has accounts | User 1 | Refused | "User 1 still has accounts. Close them first." | **Pass** |

| ID | Test | Input | Expected | Actual | Result |
|---|---|---|---|---|---|
| TC14 | Close account with money in it | Account 1 | Refused | “Account 1 still holds 1000.00 …” | **Pass** |
| TC15 | Deposit to unknown account | Account 77 | Rejected | “Account 77 does not exist.” | **Pass** |
| TC16 | History matches balance | Account 1 | Credits − debits = balance | 1,000.00 = 1,000.00 | **Pass** |

TC8 and TC12 are the most important: they show that a failed withdrawal or transfer leaves every balance exactly as it was.

# 8. Sample run

The following output was captured from a session with the text menu. Two customers were added, each was given an account (savings with 1,000.00 and checking with 250.00), and then money was moved.

### Adding a user

```
-- Add user --
  First name: Ayse
  Last name: Demir
  Age: 29
  Occupation (optional): Engineer

  User added with ID 1.
```

### Listing users

```
-- Users --
  Search by name (leave blank for all):
  ID | First name | Last name | Age | Occupation | Accounts | Total balance
  ---+------------+-----------+-----+------------+----------+--------------
  1  | Ayse       | Demir     | 29  | Engineer   | 0        | 0.00
  2  | Mehmet     | Kaya      | 41  | Teacher    | 0        | 0.00
```

### Creating an account

```
-- Create account --
  User ID: 1
    1) savings
    2) checking
    3) business
  Account type: 1
  Opening balance (blank = 0): 1000

  Account 1 created.
```

### Deposit

```
-- Deposit --
  Account ID: 1
  Amount: 500

  Deposit successful. New balance: 1,500.00
```

### Withdrawal larger than the balance (rejected) and a valid withdrawal

```
-- Withdraw --
  Account ID: 1
  Amount: 5000

  Error: Insufficient funds: account 1 has 1500.00, tried to withdraw 5000.00.
```

```
-- Withdraw --
  Account ID: 1
  Amount: 200

  Withdrawal successful. New balance: 1,300.00
```

### Transfer

```
-- Transfer --
  From account ID: 1
  To account ID: 2
  Amount: 300

  Transfer successful. Your new balance: 1,000.00
```

### Transaction history and bank summary

```
-- Transaction history --
  Account ID (leave blank for all): 1
  ID | Account | Type   | Amount   | Date
  ---+---------+--------+----------+-------------------
  5  | 1       | debit  | 300.00   | 2026-11-02 05:53:47
  4  | 1       | debit  | 200.00   | 2026-11-02 05:53:47
  3  | 1       | credit | 500.00   | 2026-11-02 05:53:47
  1  | 1       | credit | 1,000.00 | 2026-11-02 05:53:47
```

```
-- Bank summary --
  Users:          2
  Accounts:       2
  Total deposits: 1,550.00

  Type     | Accounts | Total
  ---------+----------+---------
  checking | 1        | 550.00
  savings  | 1        | 1,000.00
```

# 9. Installation and running

1. Install Python 3.9+ and MySQL 8.0.
2. Download the project: `git clone https://github.com/Yordikah/Banking-management-system.git`
3. Install the driver: `pip install -r requirements.txt`
4. Create the database: run `sql/schema.sql` in MySQL Workbench, or `mysql -u root -p < sql/schema.sql`. Optionally load sample data from `sql/seed.sql`.
5. Copy `.env.example` to `.env` and enter the MySQL password.
6. Start the program: `python main.py` (text menu) or `python main.py --gui` (window).
7. Run the tests: `python -m unittest -v`.

### Project structure

```
banking-management-system/
├── main.py                  entry point (text menu, or --gui)
├── banking/
│   ├── config.py            database settings from .env
│   ├── database.py          connection + transactional cursor
│   ├── errors.py            error types
│   ├── validation.py        input validation
│   ├── services/
│   │   ├── users.py
│   │   ├── accounts.py
│   │   └── transactions.py
│   ├── cli.py               text menu
│   └── gui.py               Tkinter window
├── sql/schema.sql           creates the database and tables
├── sql/seed.sql             sample data
├── tests/test_validation.py unit tests
├── requirements.txt
├── .env.example
└── README.md
```

# 10. Conclusion

The lab produced a working banking system on a normalised MySQL schema. The most important lesson was that checking a rule in Python is not enough when several operations can run at the same time: correctness comes from **transactions, row locks and database constraints**. Using exact DECIMAL arithmetic, parameterised queries and a separate validation layer made the program both safer and easier to test, and separating the service layer from the interfaces allowed the same logic to power both the text menu and the Tkinter window.

Possible future work: login for bank staff with hashed passwords, interest calculation for savings accounts, monthly statements exported to PDF, and integration tests that run against a dedicated MySQL test database.

# Appendix A. Full source code

The same code is available at **https://github.com/Yordikah/Banking-management-system**.

## A.1 main.py

```
"""Banking Management System entry point.

    python main.py          # text menu
    python main.py --gui    # desktop window (Tkinter)
"""

import argparse
import sys

from banking.errors import DatabaseError


def main() -> int:
    parser = argparse.ArgumentParser(description="Banking Management System (Python + MySQL)")
    parser.add_argument("--gui", action="store_true", help="open the Tkinter desktop interface")
    args = parser.parse_args()

    # Fail fast with a clear message if MySQL is unreachable.
    try:
        from banking.database import get_connection
        get_connection().close()
    except DatabaseError as err:
        print(f"{err}\nCheck your settings in the .env file (see .env.example).")
        return 1

    if args.gui:
        from banking.gui import launch
        launch()
    else:
        from banking.cli import main_menu
        try:
            main_menu()
        except (KeyboardInterrupt, EOFError):
            print("\nGoodbye!")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

## A.2 banking/config.py

```
"""Database settings, read from environment variables or a local .env file.

Credentials are never hard-coded. Copy `.env.example` to `.env` and fill it in.
"""

import os
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parent.parent


def _load_dotenv(path: Path) -> None:
    """Minimal .env loader so the project has no extra dependency for it."""
    if not path.exists():
        return
    for raw in path.read_text(encoding="utf-8").splitlines():
```

```
        line = raw.strip()
        if not line or line.startswith("#") or "=" not in line:
            continue
        key, value = line.split("=", 1)
        os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))


_load_dotenv(PROJECT_ROOT / ".env")

DB_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "port": int(os.getenv("DB_PORT", "3306")),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_NAME", "banking_app"),
}
```

## A.3 banking/errors.py

```
"""Exceptions shared by the service layer, the CLI and the GUI."""


class BankingError(Exception):
    """Base class for every error the user should see as a friendly message."""


class ValidationError(BankingError):
    """Input was missing or in the wrong format."""


class NotFoundError(BankingError):
    """A user or account id does not exist."""


class InsufficientFundsError(BankingError):
    """A withdrawal or transfer exceeds the available balance."""


class DatabaseError(BankingError):
    """MySQL could not be reached or returned an error."""
```

## A.4 banking/database.py

```
"""MySQL connection handling."""

from contextlib import contextmanager

from .config import DB_CONFIG
from .errors import DatabaseError


def get_connection():
    """Open a new MySQL connection, raising DatabaseError on failure."""
    try:
        import mysql.connector
    except ImportError as exc:  # pragma: no cover - depends on environment
        raise DatabaseError(
            "mysql-connector-python is not installed. Run: pip install -r requirements.txt"
        ) from exc

    try:
        return mysql.connector.connect(**DB_CONFIG)
    except mysql.connector.Error as err:
        raise DatabaseError(f"Could not connect to MySQL: {err}") from err
```

```python
@contextmanager
def db_cursor():
    """Yield a dictionary cursor inside one transaction.

    Commits when the block finishes, rolls back on any exception, and always
    closes the connection. Every multi-step operation (e.g. a transfer) runs
    inside a single `with db_cursor()` block so it is all-or-nothing.
    """
    conn = get_connection()
    cursor = conn.cursor(dictionary=True)
    try:
        yield cursor
        conn.commit()
    except Exception as exc:
        conn.rollback()
        _reraise_mysql_error(exc)
        raise
    finally:
        cursor.close()
        conn.close()


def _reraise_mysql_error(exc: Exception) -> None:
    """Turn raw driver errors into DatabaseError so callers handle one type."""
    try:
        import mysql.connector
    except ImportError:  # pragma: no cover
        return
    if isinstance(exc, mysql.connector.Error):
        raise DatabaseError(f"Database error: {exc}") from exc
```

## A.5 banking/validation.py

```python
"""Input parsing and validation. Pure functions, no database access."""

from decimal import Decimal, InvalidOperation

from .errors import ValidationError

ACCOUNT_TYPES = ("savings", "checking", "business")
MAX_AMOUNT = Decimal("1000000000")  # fits DECIMAL(15, 2)


def require_text(value, field: str, max_len: int = 50) -> str:
    text = (value or "").strip()
    if not text:
        raise ValidationError(f"{field} is required.")
    if len(text) > max_len:
        raise ValidationError(f"{field} must be at most {max_len} characters.")
    return text


def optional_text(value, field: str, max_len: int = 100):
    text = (value or "").strip()
    if not text:
        return None
    if len(text) > max_len:
        raise ValidationError(f"{field} must be at most {max_len} characters.")
    return text


def parse_id(value, field: str = "ID") -> int:
    try:
        number = int(str(value).strip())
    except (TypeError, ValueError):
        raise ValidationError(f"{field} must be a whole number.") from None
    if number <= 0:
```

```
        raise ValidationError(f"{field} must be positive.")
    return number


def parse_age(value) -> int:
    try:
        age = int(str(value).strip())
    except (TypeError, ValueError):
        raise ValidationError("Age must be a whole number.") from None
    if not 18 <= age <= 120:
        raise ValidationError("Age must be between 18 and 120.")
    return age


def _parse_money(value, field: str) -> Decimal:
    try:
        amount = Decimal(str(value).strip().replace(",", ""))
    except (InvalidOperation, TypeError):
        raise ValidationError(f"{field} must be a number, e.g. 250 or 99.95.") from None
    if not amount.is_finite():
        raise ValidationError(f"{field} must be a number.")
    if amount.as_tuple().exponent < -2:
        raise ValidationError(f"{field} can have at most 2 decimal places.")
    if amount >= MAX_AMOUNT:
        raise ValidationError(f"{field} is too large.")
    return amount.quantize(Decimal("0.01"))


def parse_amount(value, field: str = "Amount") -> Decimal:
    """A transaction amount: strictly greater than zero."""
    amount = _parse_money(value, field)
    if amount <= 0:
        raise ValidationError(f"{field} must be greater than zero.")
    return amount


def parse_opening_balance(value) -> Decimal:
    """An opening balance: zero or more. Blank means zero."""
    if value is None or str(value).strip() == "":
        return Decimal("0.00")
    amount = _parse_money(value, "Opening balance")
    if amount < 0:
        raise ValidationError("Opening balance cannot be negative.")
    return amount


def parse_account_type(value) -> str:
    """Accept a name ('savings') or its menu number ('1')."""
    text = (value or "").strip().lower()
    if text.isdigit() and 1 <= int(text) <= len(ACCOUNT_TYPES):
        return ACCOUNT_TYPES[int(text) - 1]
    if text in ACCOUNT_TYPES:
        return text
    options = ", ".join(f"{i}) {t}" for i, t in enumerate(ACCOUNT_TYPES, 1))
    raise ValidationError(f"Account type must be one of: {options}.")
```

## A.6 banking/services/users.py

```
"""User (customer) operations."""

from ..database import db_cursor
from ..errors import BankingError, NotFoundError
from ..validation import optional_text, parse_age, parse_id, require_text


def add_user(first_name, last_name, age, occupation=None) -> int:
    """Create a user and return the new user_id."""
```

```
    first = require_text(first_name, "First name")
    last = require_text(last_name, "Last name")
    age_value = parse_age(age)
    job = optional_text(occupation, "Occupation")

    with db_cursor() as cur:
        cur.execute(
            "INSERT INTO users (first_name, last_name, age, occupation) "
            "VALUES (%s, %s, %s, %s)",
            (first, last, age_value, job),
        )
        return cur.lastrowid


def list_users(search: str = "") -> list:
    """All users with how many accounts they hold and their total balance."""
    query = (
        "SELECT u.user_id, u.first_name, u.last_name, u.age, u.occupation, "
        "       COUNT(a.account_id) AS account_count, "
        "       COALESCE(SUM(a.balance), 0) AS total_balance "
        "FROM users u LEFT JOIN accounts a ON a.user_id = u.user_id "
    )
    params = ()
    term = (search or "").strip()
    if term:
        query += "WHERE u.first_name LIKE %s OR u.last_name LIKE %s "
        params = (f"%{term}%", f"%{term}%")
    query += "GROUP BY u.user_id ORDER BY u.user_id"

    with db_cursor() as cur:
        cur.execute(query, params)
        return cur.fetchall()


def get_user(user_id) -> dict:
    uid = parse_id(user_id, "User ID")
    with db_cursor() as cur:
        cur.execute("SELECT * FROM users WHERE user_id = %s", (uid,))
        row = cur.fetchone()
    if row is None:
        raise NotFoundError(f"User {uid} does not exist.")
    return row


def update_user(user_id, first_name, last_name, age, occupation=None) -> None:
    uid = parse_id(user_id, "User ID")
    first = require_text(first_name, "First name")
    last = require_text(last_name, "Last name")
    age_value = parse_age(age)
    job = optional_text(occupation, "Occupation")

    with db_cursor() as cur:
        cur.execute(
            "UPDATE users SET first_name = %s, last_name = %s, age = %s, occupation = %s "
            "WHERE user_id = %s",
            (first, last, age_value, job, uid),
        )
        if cur.rowcount == 0:
            cur.execute("SELECT 1 FROM users WHERE user_id = %s", (uid,))
            if cur.fetchone() is None:
                raise NotFoundError(f"User {uid} does not exist.")


def delete_user(user_id) -> None:
    """Delete a user. Refused while the user still has accounts."""
    uid = parse_id(user_id, "User ID")
    with db_cursor() as cur:
        cur.execute("SELECT 1 FROM users WHERE user_id = %s", (uid,))
```

```
        if cur.fetchone() is None:
            raise NotFoundError(f"User {uid} does not exist.")
        cur.execute("SELECT COUNT(*) AS n FROM accounts WHERE user_id = %s", (uid,))
        if cur.fetchone()["n"] > 0:
            raise BankingError(f"User {uid} still has accounts. Close them first.")
        cur.execute("DELETE FROM users WHERE user_id = %s", (uid,))
```

## A.7 banking/services/accounts.py

```
"""Bank account operations."""

from decimal import Decimal

from ..database import db_cursor
from ..errors import BankingError, NotFoundError
from ..validation import parse_account_type, parse_id, parse_opening_balance


def create_account(user_id, account_type, opening_balance="0") -> int:
    """Open an account for an existing user and return the new account_id.

    A non-zero opening balance is also recorded as a 'credit' transaction so
    the transaction history always adds up to the balance.
    """
    uid = parse_id(user_id, "User ID")
    kind = parse_account_type(account_type)
    balance = parse_opening_balance(opening_balance)

    with db_cursor() as cur:
        cur.execute("SELECT 1 FROM users WHERE user_id = %s", (uid,))
        if cur.fetchone() is None:
            raise NotFoundError(f"User {uid} does not exist. Add the user first.")

        cur.execute(
            "INSERT INTO accounts (user_id, account_type, balance) VALUES (%s, %s, %s)",
            (uid, kind, balance),
        )
        account_id = cur.lastrowid

        if balance > 0:
            cur.execute(
                "INSERT INTO transactions (account_id, transaction_type, amount) "
                "VALUES (%s, 'credit', %s)",
                (account_id, balance),
            )
        return account_id


def list_accounts(user_id=None) -> list:
    """All accounts (or one user's) with the owner's name."""
    query = (
        "SELECT a.account_id, a.user_id, "
        "       CONCAT(u.first_name, ' ', u.last_name) AS owner, "
        "       a.account_type, a.balance, a.created_at "
        "FROM accounts a JOIN users u ON u.user_id = a.user_id "
    )
    params = ()
    if user_id not in (None, ""):
        query += "WHERE a.user_id = %s "
        params = (parse_id(user_id, "User ID"),)
    query += "ORDER BY a.account_id"

    with db_cursor() as cur:
        cur.execute(query, params)
        return cur.fetchall()
```

```python
def get_account(account_id) -> dict:
    aid = parse_id(account_id, "Account ID")
    with db_cursor() as cur:
        cur.execute(
            "SELECT a.*, CONCAT(u.first_name, ' ', u.last_name) AS owner "
            "FROM accounts a JOIN users u ON u.user_id = a.user_id "
            "WHERE a.account_id = %s",
            (aid,),
        )
        row = cur.fetchone()
    if row is None:
        raise NotFoundError(f"Account {aid} does not exist.")
    return row


def close_account(account_id) -> None:
    """Close (delete) an account. Its balance must be zero."""
    aid = parse_id(account_id, "Account ID")
    with db_cursor() as cur:
        cur.execute("SELECT balance FROM accounts WHERE account_id = %s FOR UPDATE", (aid,))
        row = cur.fetchone()
        if row is None:
            raise NotFoundError(f"Account {aid} does not exist.")
        if Decimal(str(row["balance"])) != 0:
            raise BankingError(
                f"Account {aid} still holds {Decimal(str(row['balance'])):.2f}. "
                "Withdraw or transfer it before closing."
            )
        cur.execute("DELETE FROM transactions WHERE account_id = %s", (aid,))
        cur.execute("DELETE FROM accounts WHERE account_id = %s", (aid,))


def bank_summary() -> dict:
    """Totals for the dashboard / report screen."""
    with db_cursor() as cur:
        cur.execute("SELECT COUNT(*) AS users FROM users")
        users = cur.fetchone()["users"]
        cur.execute(
            "SELECT COUNT(*) AS accounts, COALESCE(SUM(balance), 0) AS deposits FROM accounts"
        )
        acc = cur.fetchone()
        cur.execute(
            "SELECT account_type, COUNT(*) AS n, COALESCE(SUM(balance), 0) AS total "
            "FROM accounts GROUP BY account_type ORDER BY account_type"
        )
        by_type = cur.fetchall()
    return {
        "users": users,
        "accounts": acc["accounts"],
        "deposits": acc["deposits"],
        "by_type": by_type,
    }
```

## A.8 banking/services/transactions.py

```python
"""Deposits, withdrawals, transfers and transaction history.

Every money movement locks the affected account rows with SELECT ... FOR UPDATE
and runs in one database transaction, so two simultaneous withdrawals can never
overdraw an account and a transfer can never half-complete.
"""

from decimal import Decimal

from ..database import db_cursor
from ..errors import InsufficientFundsError, NotFoundError, ValidationError
from ..validation import parse_amount, parse_id
```

```python
def _lock_balance(cur, account_id: int) -> Decimal:
    cur.execute("SELECT balance FROM accounts WHERE account_id = %s FOR UPDATE", (account_id,))
    row = cur.fetchone()
    if row is None:
        raise NotFoundError(f"Account {account_id} does not exist.")
    return Decimal(str(row["balance"]))


def _apply(cur, account_id: int, kind: str, amount: Decimal) -> None:
    sign = "+" if kind == "credit" else "-"
    cur.execute(
        f"UPDATE accounts SET balance = balance {sign} %s WHERE account_id = %s",
        (amount, account_id),
    )
    cur.execute(
        "INSERT INTO transactions (account_id, transaction_type, amount) VALUES (%s, %s, %s)",
        (account_id, kind, amount),
    )


def deposit(account_id, amount) -> Decimal:
    """Add money to an account. Returns the new balance."""
    aid = parse_id(account_id, "Account ID")
    value = parse_amount(amount, "Deposit amount")
    with db_cursor() as cur:
        balance = _lock_balance(cur, aid)
        _apply(cur, aid, "credit", value)
        return balance + value


def withdraw(account_id, amount) -> Decimal:
    """Take money out of an account. Returns the new balance."""
    aid = parse_id(account_id, "Account ID")
    value = parse_amount(amount, "Withdrawal amount")
    with db_cursor() as cur:
        balance = _lock_balance(cur, aid)
        if value > balance:
            raise InsufficientFundsError(
                f"Insufficient funds: account {aid} has {balance:.2f}, tried to withdraw
{value:.2f}."
            )
        _apply(cur, aid, "debit", value)
        return balance - value


def transfer(from_account_id, to_account_id, amount) -> Decimal:
    """Move money between two accounts. Returns the sender's new balance."""
    src = parse_id(from_account_id, "From account ID")
    dst = parse_id(to_account_id, "To account ID")
    value = parse_amount(amount, "Transfer amount")
    if src == dst:
        raise ValidationError("Cannot transfer to the same account.")

    with db_cursor() as cur:
        # Lock both rows in a fixed order to avoid deadlocks.
        balances = {aid: _lock_balance(cur, aid) for aid in sorted((src, dst))}
        if value > balances[src]:
            raise InsufficientFundsError(
                f"Insufficient funds: account {src} has {balances[src]:.2f}, "
                f"tried to transfer {value:.2f}."
            )
        _apply(cur, src, "debit", value)
        _apply(cur, dst, "credit", value)
        return balances[src] - value
```

```python
def get_balance(account_id) -> Decimal:
    aid = parse_id(account_id, "Account ID")
    with db_cursor() as cur:
        cur.execute("SELECT balance FROM accounts WHERE account_id = %s", (aid,))
        row = cur.fetchone()
    if row is None:
        raise NotFoundError(f"Account {aid} does not exist.")
    return Decimal(str(row["balance"]))


def history(account_id=None, limit: int = 100) -> list:
    """Most recent transactions, for one account or for the whole bank."""
    query = (
        "SELECT transaction_id, account_id, transaction_type, amount, created_at "
        "FROM transactions "
    )
    params: tuple = ()
    if account_id not in (None, ""):
        aid = parse_id(account_id, "Account ID")
        with db_cursor() as cur:
            cur.execute("SELECT 1 FROM accounts WHERE account_id = %s", (aid,))
            if cur.fetchone() is None:
                raise NotFoundError(f"Account {aid} does not exist.")
        query += "WHERE account_id = %s "
        params = (aid,)
    query += "ORDER BY created_at DESC, transaction_id DESC LIMIT %s"
    params += (int(limit),)

    with db_cursor() as cur:
        cur.execute(query, params)
        return cur.fetchall()
```

## A.9 banking/cli.py

```python
"""Text menu interface."""

from decimal import Decimal

from .errors import BankingError
from .services import accounts, transactions, users
from .validation import ACCOUNT_TYPES


# ---------------------------------------------------------------- helpers
def money(value) -> str:
    return f"{Decimal(str(value)):,.2f}"


def print_table(headers, rows) -> None:
    if not rows:
        print("  (no records)")
        return
    text_rows = [["" if v is None else str(v) for v in row] for row in rows]
    widths = [max(len(h), *(len(r[i]) for r in text_rows)) for i, h in enumerate(headers)]
    line = "-+-".join("-" * w for w in widths)
    print("  " + " | ".join(h.ljust(w) for h, w in zip(headers, widths)))
    print("  " + line)
    for r in text_rows:
        print("  " + " | ".join(v.ljust(w) for v, w in zip(r, widths)))


def ask(prompt: str) -> str:
    return input(f"  {prompt}: ").strip()


def confirm(prompt: str) -> bool:
    return ask(f"{prompt} (y/n)").lower() in ("y", "yes")
```

```
def pause() -> None:
    try:
        input("\n  Press Enter to continue...")
    except EOFError:
        pass


# ------------------------------------------------------------------ users
def add_user():
    print("\n-- Add user --")
    uid = users.add_user(
        ask("First name"), ask("Last name"), ask("Age"), ask("Occupation (optional)")
    )
    print(f"\n  User added with ID {uid}.")


def view_users():
    print("\n-- Users --")
    rows = users.list_users(ask("Search by name (leave blank for all)"))
    print_table(
        ["ID", "First name", "Last name", "Age", "Occupation", "Accounts", "Total balance"],
        [
            (r["user_id"], r["first_name"], r["last_name"], r["age"], r["occupation"],
             r["account_count"], money(r["total_balance"]))
            for r in rows
        ],
    )


def edit_user():
    print("\n-- Edit user --")
    current = users.get_user(ask("User ID"))
    print("  Leave a field blank to keep its current value.")
    first = ask(f"First name [{current['first_name']}]") or current["first_name"]
    last = ask(f"Last name [{current['last_name']}]") or current["last_name"]
    age = ask(f"Age [{current['age']}]") or current["age"]
    job = ask(f"Occupation [{current['occupation'] or ''}]") or current["occupation"]
    users.update_user(current["user_id"], first, last, age, job)
    print("\n  User updated.")


def delete_user():
    print("\n-- Delete user --")
    user = users.get_user(ask("User ID"))
    if confirm(f"Delete {user['first_name']} {user['last_name']}?"):
        users.delete_user(user["user_id"])
        print("\n  User deleted.")


# ---------------------------------------------------------------- accounts
def create_account():
    print("\n-- Create account --")
    user_id = ask("User ID")
    for i, kind in enumerate(ACCOUNT_TYPES, 1):
        print(f"    {i}) {kind}")
    kind = ask("Account type")
    opening = ask("Opening balance (blank = 0)")
    aid = accounts.create_account(user_id, kind, opening)
    print(f"\n  Account {aid} created.")


def view_accounts():
    print("\n-- Accounts --")
    rows = accounts.list_accounts(ask("User ID (leave blank for all)"))
    print_table(
        ["Account", "User", "Owner", "Type", "Balance", "Opened"],
```

```
        [
            (r["account_id"], r["user_id"], r["owner"], r["account_type"],
             money(r["balance"]), r["created_at"])
            for r in rows
        ],
    )


def close_account():
    print("\n-- Close account --")
    acc = accounts.get_account(ask("Account ID"))
    if confirm(f"Close {acc['account_type']} account {acc['account_id']} of {acc['owner']}?"):
        accounts.close_account(acc["account_id"])
        print("\n  Account closed.")


# ---------------------------------------------------------------- money
def deposit():
    print("\n-- Deposit --")
    balance = transactions.deposit(ask("Account ID"), ask("Amount"))
    print(f"\n  Deposit successful. New balance: {money(balance)}")


def withdraw():
    print("\n-- Withdraw --")
    balance = transactions.withdraw(ask("Account ID"), ask("Amount"))
    print(f"\n  Withdrawal successful. New balance: {money(balance)}")


def transfer():
    print("\n-- Transfer --")
    balance = transactions.transfer(ask("From account ID"), ask("To account ID"), ask("Amount"))
    print(f"\n  Transfer successful. Your new balance: {money(balance)}")


def check_balance():
    print("\n-- Balance --")
    acc = accounts.get_account(ask("Account ID"))
    print(f"\n  Account {acc['account_id']} ({acc['account_type']}, {acc['owner']}): "
          f"{money(acc['balance'])}")


def view_history():
    print("\n-- Transaction history --")
    rows = transactions.history(ask("Account ID (leave blank for all)"))
    print_table(
        ["ID", "Account", "Type", "Amount", "Date"],
        [
            (r["transaction_id"], r["account_id"], r["transaction_type"],
             money(r["amount"]), r["created_at"])
            for r in rows
        ],
    )


def summary():
    print("\n-- Bank summary --")
    s = accounts.bank_summary()
    print(f"  Users:          {s['users']}")
    print(f"  Accounts:       {s['accounts']}")
    print(f"  Total deposits: {money(s['deposits'])}")
    if s["by_type"]:
        print()
        print_table(
            ["Type", "Accounts", "Total"],
            [(r["account_type"], r["n"], money(r["total"])) for r in s["by_type"]],
        )
```

```python
# ---------------------------------------------------------------- menu
MENU = [
    ("Users", [
        ("Add user", add_user),
        ("View / search users", view_users),
        ("Edit user", edit_user),
        ("Delete user", delete_user),
    ]),
    ("Accounts", [
        ("Create account", create_account),
        ("View accounts", view_accounts),
        ("Check balance", check_balance),
        ("Close account", close_account),
    ]),
    ("Transactions", [
        ("Deposit money", deposit),
        ("Withdraw money", withdraw),
        ("Transfer money", transfer),
        ("Transaction history", view_history),
    ]),
    ("Reports", [
        ("Bank summary", summary),
    ]),
]


def main_menu() -> None:
    actions = {}
    while True:
        print("\n" + "=" * 44)
        print("        BANKING MANAGEMENT SYSTEM")
        print("=" * 44)
        n = 1
        for section, items in MENU:
            print(f"\n {section}")
            for label, func in items:
                print(f"   {n:>2}. {label}")
                actions[str(n)] = func
                n += 1
        print("\n    0. Exit")

        choice = input("\nEnter your choice: ").strip()
        if choice == "0":
            print("Goodbye!")
            return
        action = actions.get(choice)
        if action is None:
            print("Invalid choice. Please try again.")
            continue
        try:
            action()
        except BankingError as err:
            print(f"\n  Error: {err}")
        except (KeyboardInterrupt, EOFError):
            print("\n  Cancelled.")
        pause()
```

## A.10 banking/gui.py

```python
"""Tkinter desktop interface with Users, Accounts, Transactions and Summary tabs."""

import tkinter as tk
from decimal import Decimal
from tkinter import messagebox, ttk

from .errors import BankingError
```

```
from .services import accounts, transactions, users
from .validation import ACCOUNT_TYPES


def money(value) -> str:
    return f"{Decimal(str(value)):,.2f}"


def make_table(parent, columns, widths):
    frame = ttk.Frame(parent)
    tree = ttk.Treeview(frame, columns=[c for c, _ in columns], show="headings", height=12)
    for (key, title), width in zip(columns, widths):
        tree.heading(key, text=title)
        anchor = "e" if key in ("balance", "amount", "total_balance") else "w"
        tree.column(key, width=width, anchor=anchor)
    scroll = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
    tree.configure(yscrollcommand=scroll.set)
    tree.pack(side="left", fill="both", expand=True)
    scroll.pack(side="right", fill="y")
    frame.pack(fill="both", expand=True, padx=10, pady=(0, 10))
    return tree


def fill_table(tree, rows):
    tree.delete(*tree.get_children())
    for row in rows:
        tree.insert("", "end", values=row)


def selected_id(tree):
    sel = tree.selection()
    return tree.item(sel[0], "values")[0] if sel else None


def labeled_entry(parent, label, row, col=0, width=22):
    ttk.Label(parent, text=label).grid(row=row, column=col, sticky="w", padx=5, pady=4)
    entry = ttk.Entry(parent, width=width)
    entry.grid(row=row, column=col + 1, sticky="w", padx=5, pady=4)
    return entry


class BankingApp:
    def __init__(self, root):
        self.root = root
        root.title("Banking Management System")
        root.geometry("980x640")
        root.minsize(820, 540)

        ttk.Style().configure("Title.TLabel", font=("Segoe UI", 18, "bold"))
        ttk.Label(root, text="Banking Management System", style="Title.TLabel").pack(pady=(12, 6))

        self.notebook = ttk.Notebook(root)
        self.notebook.pack(fill="both", expand=True, padx=10, pady=10)

        self.users_tab = ttk.Frame(self.notebook)
        self.accounts_tab = ttk.Frame(self.notebook)
        self.tx_tab = ttk.Frame(self.notebook)
        self.summary_tab = ttk.Frame(self.notebook)
        self.notebook.add(self.users_tab, text="  Users  ")
        self.notebook.add(self.accounts_tab, text="  Accounts  ")
        self.notebook.add(self.tx_tab, text="  Transactions  ")
        self.notebook.add(self.summary_tab, text="  Summary  ")

        self.status = tk.StringVar(value="Ready")
        ttk.Label(root, textvariable=self.status, relief="sunken", anchor="w").pack(
            fill="x", side="bottom"
        )
```

```
        self._build_users()
        self._build_accounts()
        self._build_transactions()
        self._build_summary()
        self.notebook.bind("<<NotebookTabChanged>>", lambda _e: self.refresh_all())
        self.refresh_all()

    # ------------------------------------------------------------ plumbing
    def run(self, action, success=None):
        """Run a service call, show friendly errors, refresh the views."""
        try:
            result = action()
        except BankingError as err:
            messagebox.showerror("Error", str(err))
            self.status.set(f"Error: {err}")
            return None
        if success:
            msg = success(result) if callable(success) else success
            self.status.set(msg)
            messagebox.showinfo("Success", msg)
        self.refresh_all()
        return result

    def refresh_all(self):
        try:
            self._refresh_users()
            self._refresh_accounts()
            self._refresh_transactions()
            self._refresh_summary()
        except BankingError as err:
            self.status.set(f"Error: {err}")

    # ------------------------------------------------------------ users tab
    def _build_users(self):
        form = ttk.LabelFrame(self.users_tab, text="User details")
        form.pack(fill="x", padx=10, pady=10)
        self.u_first = labeled_entry(form, "First name", 0, 0)
        self.u_last = labeled_entry(form, "Last name", 0, 2)
        self.u_age = labeled_entry(form, "Age", 1, 0)
        self.u_job = labeled_entry(form, "Occupation", 1, 2)

        buttons = ttk.Frame(form)
        buttons.grid(row=2, column=0, columnspan=4, sticky="w", padx=5, pady=6)
        ttk.Button(buttons, text="Add user", command=self.add_user).pack(side="left", padx=3)
        ttk.Button(buttons, text="Update selected", command=self.update_user).pack(side="left",
padx=3)
        ttk.Button(buttons, text="Delete selected", command=self.delete_user).pack(side="left",
padx=3)
        ttk.Button(buttons, text="Clear", command=self.clear_user_form).pack(side="left", padx=3)

        search = ttk.Frame(self.users_tab)
        search.pack(fill="x", padx=10)
        ttk.Label(search, text="Search:").pack(side="left")
        self.u_search = ttk.Entry(search, width=30)
        self.u_search.pack(side="left", padx=5)
        self.u_search.bind("<KeyRelease>", lambda _e: self._refresh_users())

        self.users_table = make_table(
            self.users_tab,
            [("id", "ID"), ("first", "First name"), ("last", "Last name"), ("age", "Age"),
             ("job", "Occupation"), ("n", "Accounts"), ("total_balance", "Total balance")],
            [50, 140, 140, 60, 170, 80, 130],
        )
        self.users_table.bind("<<TreeviewSelect>>", self._on_user_select)

    def _refresh_users(self):
        rows = users.list_users(self.u_search.get())
        fill_table(self.users_table, [
```

```
            (r["user_id"], r["first_name"], r["last_name"], r["age"], r["occupation"] or "",
             r["account_count"], money(r["total_balance"]))
            for r in rows
        ])

    def _on_user_select(self, _event):
        values = self.users_table.item(self.users_table.selection()[0], "values") \
            if self.users_table.selection() else None
        if not values:
            return
        for entry, value in zip((self.u_first, self.u_last, self.u_age, self.u_job), values[1:5]):
            entry.delete(0, tk.END)
            entry.insert(0, value)

    def clear_user_form(self):
        for entry in (self.u_first, self.u_last, self.u_age, self.u_job):
            entry.delete(0, tk.END)
        self.users_table.selection_remove(self.users_table.selection())

    def add_user(self):
        uid = self.run(
            lambda: users.add_user(self.u_first.get(), self.u_last.get(),
                                   self.u_age.get(), self.u_job.get()),
            lambda new_id: f"User added with ID {new_id}.",
        )
        if uid:
            self.clear_user_form()

    def update_user(self):
        uid = selected_id(self.users_table)
        if not uid:
            messagebox.showwarning("Update user", "Select a user in the table first.")
            return
        self.run(
            lambda: users.update_user(uid, self.u_first.get(), self.u_last.get(),
                                      self.u_age.get(), self.u_job.get()),
            f"User {uid} updated.",
        )

    def delete_user(self):
        uid = selected_id(self.users_table)
        if not uid:
            messagebox.showwarning("Delete user", "Select a user in the table first.")
            return
        if messagebox.askyesno("Delete user", f"Delete user {uid}?"):
            self.run(lambda: users.delete_user(uid), f"User {uid} deleted.")
            self.clear_user_form()

    # ------------------------------------------------------------ accounts tab
    def _build_accounts(self):
        form = ttk.LabelFrame(self.accounts_tab, text="Open a new account")
        form.pack(fill="x", padx=10, pady=10)

        ttk.Label(form, text="Owner").grid(row=0, column=0, sticky="w", padx=5, pady=4)
        self.a_owner = ttk.Combobox(form, width=30, state="readonly")
        self.a_owner.grid(row=0, column=1, sticky="w", padx=5, pady=4)

        ttk.Label(form, text="Type").grid(row=0, column=2, sticky="w", padx=5, pady=4)
        self.a_type = ttk.Combobox(form, values=ACCOUNT_TYPES, width=14, state="readonly")
        self.a_type.current(0)
        self.a_type.grid(row=0, column=3, sticky="w", padx=5, pady=4)

        self.a_opening = labeled_entry(form, "Opening balance", 1, 0)
        self.a_opening.insert(0, "0")

        buttons = ttk.Frame(form)
        buttons.grid(row=2, column=0, columnspan=4, sticky="w", padx=5, pady=6)
        ttk.Button(buttons, text="Create account", command=self.create_account).pack(side="left",
```

```
padx=3)
        ttk.Button(buttons, text="Close selected account",
command=self.close_account).pack(side="left", padx=3)

        self.accounts_table = make_table(
            self.accounts_tab,
            [("id", "Account"), ("uid", "User"), ("owner", "Owner"), ("type", "Type"),
             ("balance", "Balance"), ("opened", "Opened")],
            [70, 60, 200, 110, 130, 170],
        )

    def _refresh_accounts(self):
        all_users = users.list_users()
        self._owner_ids = [u["user_id"] for u in all_users]
        self.a_owner["values"] = [
            f"{u['user_id']} - {u['first_name']} {u['last_name']}" for u in all_users
        ]
        rows = accounts.list_accounts()
        fill_table(self.accounts_table, [
            (r["account_id"], r["user_id"], r["owner"], r["account_type"],
             money(r["balance"]), r["created_at"])
            for r in rows
        ])
        self._account_choices = [
            f"{r['account_id']} - {r['owner']} ({r['account_type']})" for r in rows
        ]

    def create_account(self):
        idx = self.a_owner.current()
        if idx < 0:
            messagebox.showwarning("Create account", "Choose the account owner first.")
            return
        user_id = self._owner_ids[idx]
        self.run(
            lambda: accounts.create_account(user_id, self.a_type.get(), self.a_opening.get()),
            lambda aid: f"Account {aid} created.",
        )

    def close_account(self):
        aid = selected_id(self.accounts_table)
        if not aid:
            messagebox.showwarning("Close account", "Select an account in the table first.")
            return
        if messagebox.askyesno("Close account", f"Close account {aid}? Its history is removed."):
            self.run(lambda: accounts.close_account(aid), f"Account {aid} closed.")

    # ------------------------------------------------------------ transactions tab
    def _build_transactions(self):
        form = ttk.LabelFrame(self.tx_tab, text="Move money")
        form.pack(fill="x", padx=10, pady=10)

        ttk.Label(form, text="Account").grid(row=0, column=0, sticky="w", padx=5, pady=4)
        self.t_account = ttk.Combobox(form, width=36)
        self.t_account.grid(row=0, column=1, sticky="w", padx=5, pady=4)
        self.t_account.bind("<<ComboboxSelected>>", lambda _e: self._refresh_transactions())

        ttk.Label(form, text="Transfer to").grid(row=1, column=0, sticky="w", padx=5, pady=4)
        self.t_target = ttk.Combobox(form, width=36)
        self.t_target.grid(row=1, column=1, sticky="w", padx=5, pady=4)

        self.t_amount = labeled_entry(form, "Amount", 0, 2, width=14)

        buttons = ttk.Frame(form)
        buttons.grid(row=2, column=0, columnspan=4, sticky="w", padx=5, pady=6)
        ttk.Button(buttons, text="Deposit", command=self.deposit).pack(side="left", padx=3)
        ttk.Button(buttons, text="Withdraw", command=self.withdraw).pack(side="left", padx=3)
        ttk.Button(buttons, text="Transfer", command=self.transfer).pack(side="left", padx=3)
        ttk.Button(buttons, text="Show all history",
```

```
                command=self.show_all_history).pack(side="left", padx=12)

        self.t_balance = tk.StringVar(value="")
        ttk.Label(self.tx_tab, textvariable=self.t_balance,
                  font=("Segoe UI", 11, "bold")).pack(anchor="w", padx=12)

        self.tx_table = make_table(
            self.tx_tab,
            [("id", "ID"), ("acc", "Account"), ("type", "Type"), ("amount", "Amount"),
             ("date", "Date")],
            [60, 80, 90, 130, 180],
        )

    @staticmethod
    def _id_from_choice(text):
        return text.split(" - ", 1)[0].strip() if text else ""

    def _refresh_transactions(self):
        choices = getattr(self, "_account_choices", [])
        self.t_account["values"] = choices
        self.t_target["values"] = choices
        aid = self._id_from_choice(self.t_account.get())
        try:
            rows = transactions.history(aid or None)
            self.t_balance.set(
                f"Balance of account {aid}: {money(transactions.get_balance(aid))}" if aid
                else "All accounts (latest 100 transactions)"
            )
        except BankingError as err:
            rows = []
            self.t_balance.set(str(err))
        fill_table(self.tx_table, [
            (r["transaction_id"], r["account_id"], r["transaction_type"],
             money(r["amount"]), r["created_at"])
            for r in rows
        ])

    def show_all_history(self):
        self.t_account.set("")
        self._refresh_transactions()

    def _money_action(self, func, verb):
        aid = self._id_from_choice(self.t_account.get())
        result = self.run(
            lambda: func(aid, self.t_amount.get()),
            lambda bal: f"{verb} successful. New balance: {money(bal)}",
        )
        if result is not None:
            self.t_amount.delete(0, tk.END)

    def deposit(self):
        self._money_action(transactions.deposit, "Deposit")

    def withdraw(self):
        self._money_action(transactions.withdraw, "Withdrawal")

    def transfer(self):
        src = self._id_from_choice(self.t_account.get())
        dst = self._id_from_choice(self.t_target.get())
        result = self.run(
            lambda: transactions.transfer(src, dst, self.t_amount.get()),
            lambda bal: f"Transfer successful. New balance of account {src}: {money(bal)}",
        )
        if result is not None:
            self.t_amount.delete(0, tk.END)

    # ------------------------------------------------------------ summary tab
    def _build_summary(self):
```

```
        box = ttk.Frame(self.summary_tab)
        box.pack(fill="x", padx=20, pady=20)
        self.s_users = tk.StringVar()
        self.s_accounts = tk.StringVar()
        self.s_deposits = tk.StringVar()
        for col, (title, var) in enumerate(
            (("Users", self.s_users), ("Accounts", self.s_accounts),
             ("Total deposits", self.s_deposits))
        ):
            card = ttk.LabelFrame(box, text=title)
            card.grid(row=0, column=col, padx=10, sticky="nsew")
            ttk.Label(card, textvariable=var, font=("Segoe UI", 20, "bold")).pack(padx=30, pady=15)
            box.columnconfigure(col, weight=1)

        ttk.Label(self.summary_tab, text="By account type").pack(anchor="w", padx=12)
        self.summary_table = make_table(
            self.summary_tab,
            [("type", "Type"), ("n", "Accounts"), ("total_balance", "Total balance")],
            [160, 100, 160],
        )

    def _refresh_summary(self):
        s = accounts.bank_summary()
        self.s_users.set(str(s["users"]))
        self.s_accounts.set(str(s["accounts"]))
        self.s_deposits.set(money(s["deposits"]))
        fill_table(self.summary_table, [
            (r["account_type"], r["n"], money(r["total"])) for r in s["by_type"]
        ])


def launch() -> None:
    root = tk.Tk()
    BankingApp(root)
    root.mainloop()
```

## A.11 tests/test_validation.py

```
"""Unit tests for input validation. No database needed:  python -m unittest"""

import unittest
from decimal import Decimal

from banking.errors import ValidationError
from banking.validation import (
    parse_account_type,
    parse_age,
    parse_amount,
    parse_id,
    parse_opening_balance,
    require_text,
)


class TextTests(unittest.TestCase):
    def test_trims(self):
        self.assertEqual(require_text("  Ali ", "Name"), "Ali")

    def test_blank_rejected(self):
        with self.assertRaises(ValidationError):
            require_text("   ", "Name")

    def test_too_long_rejected(self):
        with self.assertRaises(ValidationError):
            require_text("x" * 51, "Name")
```

```python
class NumberTests(unittest.TestCase):
    def test_ids(self):
        self.assertEqual(parse_id(" 7 "), 7)
        for bad in ("0", "-3", "abc", "", None, "1.5"):
            with self.assertRaises(ValidationError):
                parse_id(bad)

    def test_age(self):
        self.assertEqual(parse_age("30"), 30)
        for bad in ("17", "121", "thirty"):
            with self.assertRaises(ValidationError):
                parse_age(bad)


class MoneyTests(unittest.TestCase):
    def test_amount_valid(self):
        self.assertEqual(parse_amount("100"), Decimal("100.00"))
        self.assertEqual(parse_amount("1,250.5"), Decimal("1250.50"))

    def test_amount_rejects_zero_negative_and_junk(self):
        for bad in ("0", "-5", "abc", "", "NaN", "Infinity", "10.999"):
            with self.assertRaises(ValidationError, msg=bad):
                parse_amount(bad)

    def test_amount_rejects_huge(self):
        with self.assertRaises(ValidationError):
            parse_amount("7500000000000")

    def test_opening_balance(self):
        self.assertEqual(parse_opening_balance(""), Decimal("0.00"))
        self.assertEqual(parse_opening_balance("0"), Decimal("0.00"))
        self.assertEqual(parse_opening_balance("50.25"), Decimal("50.25"))
        with self.assertRaises(ValidationError):
            parse_opening_balance("-1")


class AccountTypeTests(unittest.TestCase):
    def test_by_name_and_number(self):
        self.assertEqual(parse_account_type("Savings"), "savings")
        self.assertEqual(parse_account_type("2"), "checking")
        self.assertEqual(parse_account_type("3"), "business")

    def test_invalid(self):
        for bad in ("", "gold", "4", "0"):
            with self.assertRaises(ValidationError):
                parse_account_type(bad)


if __name__ == "__main__":
    unittest.main()
```

## A.12 sql/schema.sql

```sql
-- Banking Management System - database schema (MySQL 8+)
-- Run with:  mysql -u root -p < sql/schema.sql

CREATE DATABASE IF NOT EXISTS banking_app;
USE banking_app;

CREATE TABLE IF NOT EXISTS users (
    user_id     INT AUTO_INCREMENT PRIMARY KEY,
    first_name  VARCHAR(50)  NOT NULL,
    last_name   VARCHAR(50)  NOT NULL,
    age         INT          NOT NULL,
    occupation  VARCHAR(100),
    CONSTRAINT chk_users_age CHECK (age BETWEEN 18 AND 120)
);
```

```sql
CREATE TABLE IF NOT EXISTS accounts (
    account_id    INT AUTO_INCREMENT PRIMARY KEY,
    user_id       INT            NOT NULL,
    account_type  VARCHAR(20)    NOT NULL,
    balance       DECIMAL(15, 2) NOT NULL DEFAULT 0.00,
    created_at    DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_accounts_user FOREIGN KEY (user_id) REFERENCES users (user_id),
    CONSTRAINT chk_accounts_balance CHECK (balance >= 0),
    CONSTRAINT chk_accounts_type CHECK (account_type IN ('savings', 'checking', 'business'))
);

CREATE TABLE IF NOT EXISTS transactions (
    transaction_id    INT AUTO_INCREMENT PRIMARY KEY,
    account_id        INT            NOT NULL,
    transaction_type  VARCHAR(10)    NOT NULL,
    amount            DECIMAL(15, 2) NOT NULL,
    created_at        DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transactions_account FOREIGN KEY (account_id) REFERENCES accounts (account_id),
    CONSTRAINT chk_transactions_type CHECK (transaction_type IN ('credit', 'debit')),
    CONSTRAINT chk_transactions_amount CHECK (amount > 0),
    INDEX idx_transactions_account (account_id, created_at)
);
```

## A.13 sql/seed.sql

```sql
-- Optional sample data. Run after schema.sql:
--   mysql -u root -p banking_app < sql/seed.sql

INSERT INTO users (first_name, last_name, age, occupation) VALUES
    ('Ayşe',  'Demir',   29, 'Engineer'),
    ('Mehmet','Kaya',    41, 'Teacher'),
    ('Elif',  'Yılmaz',  23, 'Student');

INSERT INTO accounts (user_id, account_type, balance) VALUES
    (1, 'checking', 1500.00),
    (1, 'savings',  5000.00),
    (2, 'checking',  800.00),
    (3, 'savings',   250.00);

INSERT INTO transactions (account_id, transaction_type, amount) VALUES
    (1, 'credit', 1500.00),
    (2, 'credit', 5000.00),
    (3, 'credit',  800.00),
    (4, 'credit',  250.00);
```